

CURSO C#

1. TEMARIO

2-12 | SINTAXIS

13-22 | POO

55-87 | CARACTERÍSTICAS AVANZADAS

88-127 | CONSTRUCCIÓN DE APLICACIONES

2. SINTAXIS BÁSICA 1

Comentarios:

// Esto es un comentario

/* Esto también lo es */

3. SINTAXIS BÁSICA 2

Tipos de datos:

<u>int = ENTERO</u>	<u>string = CADENA</u>	<u>float = FLOTANTE</u>	<u>bool = BOOLEANO</u>
24 ; 42 ; -12 ; 102	"hola"	28,2 ; 14,3 ; 34,4 ; -32,32	true ; false

Variables:

Declaración:

(tipo de dato) (nombre de la variable) = (dato)

Variable implícitas:

var (nombre de la variable) = (dato)

Constantes:

constante = variable

excepto por... Las constantes no se pueden modificar/redeclarar

4. SINTAXIS BÁSICA 3

Operador	Descripción	Ejemplo
+	Suma	<code>>>> 3 + 2</code> 5
-	Resta	<code>>>> 4 - 7</code> -3
-	Negación	<code>>>> -7</code> -7
*	Multiplicación	<code>>>> 2 * 6</code> 12
**	Exponente	<code>>>> 2 ** 6</code> 64
/	División	<code>>>> 3.5 / 2</code> 1.75
//	División entera	<code>>>> 3.5 // 2</code> 1.0
%	Módulo	<code>>>> 7 % 2</code> 1

5. SINTAXIS BÁSICA FINAL

Conversión:

```
string datoSinConvertir = "12"
int datoConvertido = int.Parse(datoSinConvertir)
```

6. FUNCIONES

Funcion:

```
(tipo de dato)(nombre de variable)((parametros)){
    (acciones)
```

```
}
```

`void` = no return dato

7. FUNCIONES 2

Sobrecarga: Las funciones se pueden sobrecargar para admitir distintos tipos de datos y distinta cantidad de parámetros

8. FUNCIONES 3

Parámetros Opcionales: podemos poner parámetros opcionales declarándose en la zona de parámetros

9. IF

```
If (condición){  
    (acción)  
}else if(condición){  
    (acción)  
}else{  
    (acción)  
}
```

10.SWITCH

Agarra una variable y si el dato coincide con alguno de los casos, lo realiza

```
switch (algo){  
    case "caso1":  
        (accion)  
    default:
```

```
        (acción default)
    }
```

11.WHILE

```
while (código a evaluar){
    código a repetir
}
do{
    código a repetir
}while(código a evaluar)
```

son lo mismo pero el do while se ejecuta si o si una vez

12.EXCEPCIONES

```
try{
    intentando código
}
catch (exception ex){
    hacer algo
}
finally {
    hacer algo mas si o si
}
```

catcher general (exception), catcher general en caso de no saber los nombres:

```
(exception e) when (e.GetType() != typeof(FormatException))
```

checked, comprueba los más mínimos errores por que a veces el editor de código lo ignora

lanzador de excepciones: throw

13. POO (CREACIONES)

Creando clase:

```
class Clase{
```

Creando propiedades estáticas:

```
class Clase{  
    string propiedad1 = "propiedad"  
}
```

Creando propiedades con constructor propio:

```
class Clase{  
    string propiedad1;  
  
    public Clase(parametro1){  
        this.propiedad1 = parametro1  
    }  
}
```

Creando Métodos:

```
class Clase{  
    string metodo1(){  
        // MÉTODO  
    }  
}
```

```
}
```

14. POO (ENCAPSULACIÓN)

Encapsulación:

public: accesible desde todo el documento

protected: accesible desde la misma clase y clases heredadas

private: accesible solo desde la clase (necesitando un getter)

Getter y Setter:

```
setAlgo(nuevo){  
    antiguo = nuevo  
}  
getAlgo(){  
    return algo  
}
```

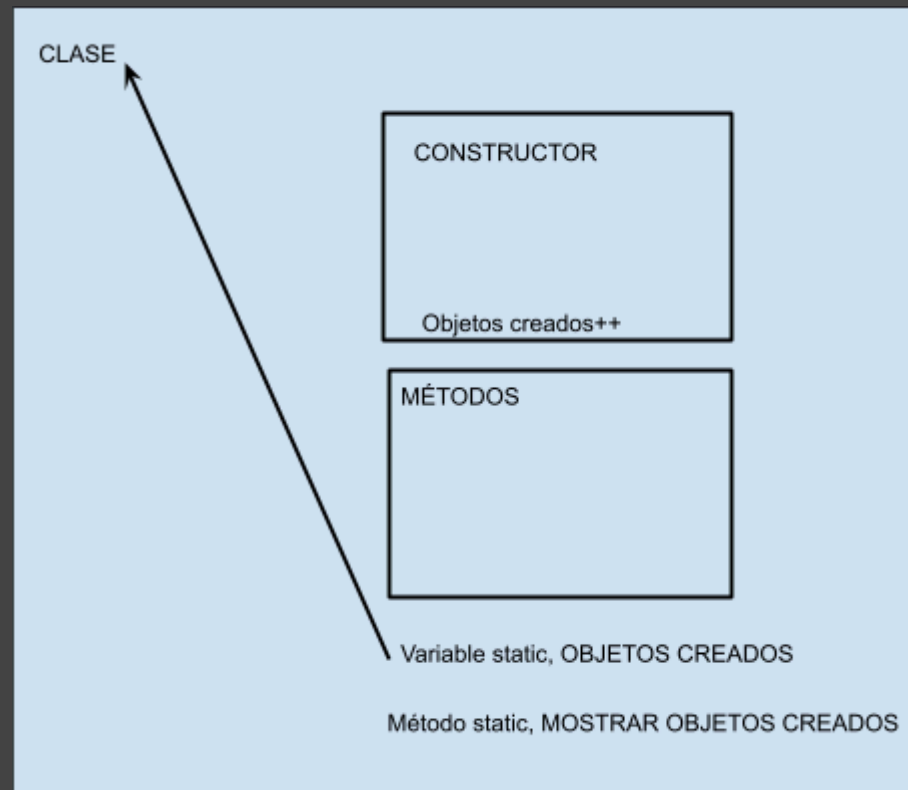
15. POO (STATIC)

Imagina que una clase es como una caja de juguetes y cada vez que haces una nueva caja, obtienes una copia de todos los juguetes. Ahora, si tienes un juguete que todos deberían compartir, como un balón de fútbol, lo haces estático. Así, en lugar de tener un balón para cada caja, todos comparten el mismo balón, ¡como si estuvieran jugando juntos en un solo equipo sin tener que hacer una caja nueva cada vez! ¡Eso es lo que significa "static" en C#!

En POO, las clases son como planos para crear objetos. El uso de la palabra clave "static" en C# Está relacionado con la forma en que algunos elementos (métodos, campos, propiedades) de esas clases se comportan.

Cuando algo es estático, significa que pertenece a la clase en sí y no a las instancias individuales de esa clase. En cambio, los elementos no estáticos suelen pertenecer a cada objeto que creas a partir de la clase.

Entonces, "static" en C# te permite definir cosas que son comunes a toda la clase y no dependen de instancias particulares. Es una forma de organizar y compartir información entre todas las instancias de la clase.



Un ejemplo de uso es este, se crea una variable static objetos creados y se aumenta cada vez que se crea un objeto.

si no fuera static se le podría llamar desde cualquier objeto y encima daría 1 siempre

pero como es static sólo se puede llamar referenciándose a la clase Clase.ObjetosCreados

y a demás como es uno solo cuando llamemos a mostrar se van a ver los objetos creados

16. POO (CLASES ANÓNIMAS)

```
var NombreClase = new { Nombre = 'fsdfwd' , Edad = 31 }
```

acceso: NombreClase.Nombre

17. POO (HERENCIA)

```
class Clase : ClasePadre
```

```
    public Clase(string nombre) : base(nombre)
```

18. POO (POLIMORFISMO)

```
void virtual metodoACambiar{
```

```
    a
```

```
}
```

```
void override metodoACambiar{
```

```
    b
```

```
}
```

19. POO (INTERFACES)

Las interfaces son una especie de libro de reglas que deben seguir todas las clases de un mismo tipo, dentro de una interfaz se declaran métodos sin ninguna función, y cuando se hereda una interfaz a una clase, la clase está obligada a completar esos métodos

sintaxis:

```
interface Interfaz{
```

```
    int MetodoObligatorio();
```

```
}
```

interfaces con mismos métodos: Para especificar qué método estamos queriendo declarar dentro de la clase la fórmula es:

tipo de dato + nombre interfaz + . + método + {}

para acceder al método desde el main declaramos una variable tipo (la interfaz que tiene el método) y como valor le damos el objeto de la clase que usa su método, luego refiriéndonos a la variable Ivariable llamamos al método

20. POO (CLASES ABSTRACTAS)

para obligar al programador a incluir métodos en clases herederas de una clase madre/abstracta se hace así

```
abstract class Clase{  
    public abstract void MetodoObligatorio();  
}
```

```
class Heredera : Clase{  
    public override void MetodoObligatorio();  
}
```

21. POO (CLASES SELLADAS)

para evitar que se creen clases herederas de una clase, se usa la sintaxis:

```
sealed class Mono{  
}
```

Método:

```
public sealed string Hola(){  
}
```

22. POO (PROPERTIES)

Las properties son un getter combinado con un setter que actúa como variable en las propiedades de un objeto

```
private double evaluaSalario(double salario)  
{  
    if (salario < 0) return 0;  
    else return salario;  
}  
  
// CREACIÓN DE PROPIEDAD  
  
public double SALARIO  
{  
    get { return this.salario; }  
    set { this.salario=evaluaSalario(value); }  
}
```

23. ARRAYS

Formas de declararlos:

1.{

 tipo de dato + [] + nombre array

 nombre array + = + new + tipo de dato + [cantidad de elementos]

 nombre array[numero elemento] = dato

}

2.{

 tipo de dato + [] + nombre array + = + new + tipo de dato + [cantidad de elementos]

 nombre array[numero elemento] = dato

}

3.{

 tipo de dato + [] + nombre array + = + {dato1,dato2,dato3}

}

4. Implícito {

 var + [] + nombre array + = + {dato1,dato2,dato3}

}

5. Array de Clases Anónimas{

 var personas = new[]{

 new{ Nombre = 'fsdfwd', Edad = 31}

 }

}

6

24. FOR

```
for( int i = 0; i<10; i++ ){  
    código a repetir  
}
```

for each:

```
for(var persona in personas){  
    código a repetir  
}
```

25. ENUM

enum es una lista de valores constantes

```
enum hola = {enero,febrero,marzo...}
```

se accede a ellas con hola.enero

```
enum hola = {enero = 5005, febrero = 200}  
para acceder a los datos hay que hacer casting  
hola amigo = hola.enero  
double eneroAmigo = (double)amigo
```

26. COLLECTIONS

Se les llama con System.Collections.Generic

Lista (Array ordenable, modificable)

```
List<Tipo de dato> NombreLista = new List<Tipo de Dato>();
```

```
NombreLista.Add(DATO);
```

```
NombreLista.Count();
```

Lista Enlazada (Lista variable optimizada)

Las linked list se construyen por nodos, cada elemento tiene 2 conexiones, una con el elemento anterior y otra con el siguiente, en una lista normal si sacaramos un elemento del medio, todos los elementos siguientes tendrían que moverse un lugar hacia arriba, en una linkedlist al eliminar un elemento del medio, se actualizan las conexiones de los nodos afectados y listo, es decir se modifican 2 conexiones nada más

```
LinkedList<Tipo de dato> NombreLista = new LinkedList<Tipo de Dato>();
```

```
NombreLista.AddFirst(DATO)
```

```
NombreLista.AddLast(DATO)
```

```
NombreLista.Clear()
```

Queue (F.I.F.O) First In First Out, el primero que entró es el primero que se va como en una cola de super

```
Queue<Tipo de dato> NombreLista = new Queue<Tipo de Dato>();
```

```
NombreLista.Enqueue(DATO)
```

NombreLista.Dequeue(DATO)

Stack (L.I.F.O) Last In First Out, el ultimo que entro es el primero que se va

```
Stack<Tipo de dato> NombreLista = new Stack<Tipo de Dato>();
```

NombreLista.Push(DATO) pone el dato, y al siguiente que venga lo pone adelante, es decir el primero que puso va a ser el último

NombreLista.Pop() Saca el último

Dictionary: Es como un array pero cada elemento tiene una key y un value, Ej: Contraseña : Hola.23

```
Dictionary<tipodato del Key, tipodato del Value> NombreLista = new Dictionary<tipodato del Key, tipodato del Value>();
```

```
NombreLista.Add("ContraseñaYT", "contrapro23");
```

```
NombreLista["ConstraseñaTW"]="contrapro";
```

```
foreach(KeyValuePair<tipodato del Key, tipodato del Value> persona in NombreLista) {
```

```
    Console.WriteLine(NombreLista.Key + " " + NombreLista.Value
```

```
}
```